
SovereignAI

Project Review and Production Roadmap

What is implemented, how it works, what has been measured, and what remains

Repository-evidence snapshot - 1 October 2026

Advisory industrial AI prototype; not a production or safety certification.

Contents

Executive assessment	3
Evidence tiers used in this report	3
1. System architecture and how it was built	3
1.1 Ownership and trust boundaries	3
1.2 Development sequence	4
2. Implemented components: mechanism and limits	4
2.1 Local inference and bounded execution	4
2.2 Evidence retrieval and source permissions	4
2.3 Industrial diagnostics and deterministic calculations	5
2.4 ControlPlane and factuality verification	5
2.5 Workflows, organizations, approval, and outputs	5
2.6 Audit trail and Evidence Capsules	5
3. What has actually been measured	6
3.1 Test and integration evidence	6
3.2 Retrieval observations	6
3.3 ControlPlane observations	6
3.4 Local inference observations and the incomplete new plan	6
4. Equations and what they mean here	6
4.1 Implemented retrieval fusion	6
4.2 Illustrative engineering threshold calculation	7
4.3 Recommended factuality and release metrics	7
4.4 Implemented capsule identity	7
5. Production blockers and why they matter	7
6. Step-by-step route to a production-quality supervised pilot	8
Phase 0 — Freeze scope and risk contract	8
Phase 1 — Close factuality and authorization gaps	8
Phase 2 — Build an independent evaluation corpus	8
Phase 3 — Run a controlled measurement program	9
Phase 4 — Run the full Pump-102 assurance exercise	9
Phase 5 — Harden infrastructure and software delivery	9
Phase 6 — Shadow, supervise, then reconsider scope	10
7. Immediate work order	10
Source and terminology register	10
Local repository evidence	10
External primary references	11

Status and scope. This is a point-in-time engineering report for the local checkout at commit de965a9 on branch pump102-inference-completion. It is not a regulatory assessment, penetration test, independent safety certification, or claim that the system is production-ready. The untracked experiments/results/inference_v3_plan/summary.json was inspected as a request-free plan, not as benchmark evidence. No environment files, private credentials, or customer data were inspected. The three maintained project documents remain docs/CURRENT_FEATURES.md, docs/PAST_EXPERIMENTS.md, and docs/NEXT_PLAN.md. This report is a dated snapshot, not a fourth source of truth. Source identifiers in square brackets refer to the register at the end.

Executive assessment

SovereignAI aims to provide a **private, authorized, evidence-backed industrial assistant**. An engineer should be able to ask about equipment such as Pump-102, combine relevant manuals, maintenance history, and measured sensor evidence, receive an advisory recommendation with provenance, and retain an auditable record. The design explicitly excludes autonomous physical control: it must not issue PLC, DCS, SCADA, alarm, setpoint, or equipment commands. Qualified people retain responsibility for consequential decisions. [L1]

The repository contains a working local-first prototype with a FastAPI and Next.js workbench, Ollama/vLLM inference adapters, authorization-aware retrieval, a bounded tool agent, a Graph-RAG integration, a separate time-series diagnostic service, a ControlPlane release gate, organization/workflow packs, real document artifacts, and integrity-checkable Evidence Capsules. Many individual and integration contracts are tested. A **fully live, networked Pump-102 assurance run with all services and independently reviewed evidence has not yet been completed**. The newest controlled inference harness has engineering controls and tests, but its final live comparison has not run. [L1-L4]

The strongest next move is **assurance, not feature expansion**: close ControlPlane coverage and release-policy gaps, obtain a properly reviewed evidence set, evaluate retrieval and diagnostics without leakage, run the whole system under success and failure conditions, then harden identity, data, operations, and deployment for a read-only supervised pilot. The current prototype should be presented as advisory research software, not a certified hallucination detector or plant-control system. [L1-L3, L8-L10]

Evidence tiers used in this report

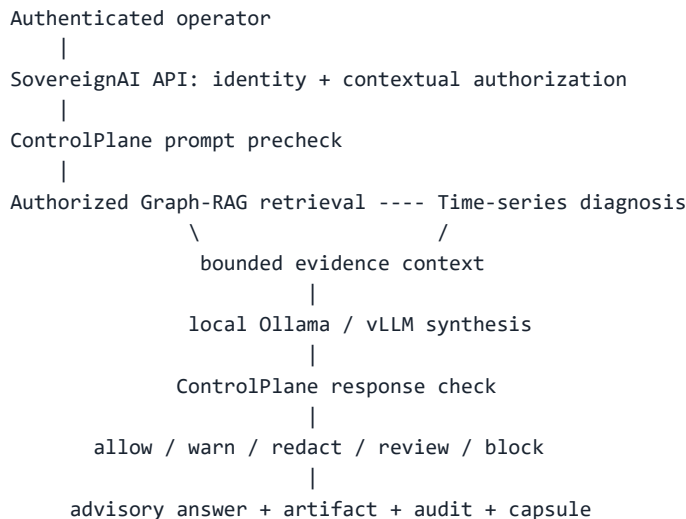
Tier	Meaning	Examples in this checkout
Implemented	Code and configuration exist	ACLs, Workcell Packs, capsules, inference harness
Contract-tested	Tests simulate a component or boundary	Seven Pump-102 integration cases, 83 focused inference-harness tests
Locally observed	A real local component was exercised	Preliminary Ollama/vLLM requests; ControlPlane synthetic evaluation
Field-validated	Independently reviewed industrial evidence and live end-to-end operation	Not yet established

This distinction matters: a passing fixture test establishes the contract under that fixture. It does not establish effectiveness on unseen plant data.

1. System architecture and how it was built

1.1 Ownership and trust boundaries

The host application, SovereignAI, owns authentication, role and resource authorization, orchestration, approvals, audit, and artifact delivery. Graph-RAG supplies documentary evidence. The Time-Series Diagnostic Agent supplies structured sensor findings. A configured local Ollama or vLLM runtime generates text. ControlPlane performs prompt precheck and response-release policy checks. This separation prevents a language model from silently becoming the authorization service, numerical calculator, or plant actuator. [L1, L5-L9]



The integrated orchestrator first constructs an effective access scope and calls the ControlPlane precheck. It then invokes the requested Graph-RAG and diagnostic services, builds a bounded prompt, generates a candidate locally, and submits the response with evidence to ControlPlane. A blocked or human-review action is not released. Required-service failures are reported rather than fabricated as successful results. The networked full path remains an outstanding validation milestone. [L5, L2]

1.2 Development sequence

The repository progressed from a local API/workbench and agent/tool baseline through streaming and resource management, hybrid retrieval and typed evidence, Workcell Packs and capsules, local identity and ACLs, asset context, Organization Packs, then three-service integration contracts. Later work added a controlled inference-comparison harness and factuality-state refinements. Historical test counts in the project log rise from 43 through 134 backend tests across these batches; they are historical checkpoints, not a current single-suite pass count. [L2]

This sequence matters because the assurance boundary was added around a capability-rich prototype. The next stage should stabilize and prove that boundary rather than add another autonomous agent. [L3]

2. Implemented components: mechanism and limits

2.1 Local inference and bounded execution

Model calls pass through a provider-neutral interface. Configuration assigns GENERAL, VISION, and CODER roles, while Ollama or vLLM serves local generation. The workbench supports FAST, STANDARD, and DEEP execution, streaming, cancellation, timeouts, and readiness checks by model role. The agent follows a bounded plan/act/observe/verify cycle and may call only registered, schema-validated tools. Generated Python executes in a restricted Docker container with no network, a read-only root filesystem, dropped capabilities, limited CPU/memory/processes, and no host-execution fallback. If Docker is unavailable, execution is reported unavailable. [L1, L11]

What this achieves: model and tool failures are distinguishable from a successful answer. **What remains:** host-specific operational testing, capacity planning, dependency health, queueing, and rollback under load.

2.2 Evidence retrieval and source permissions

The backend combines dense semantic retrieval and sparse BM25 ranking using reciprocal-rank fusion (RRF). An optional local reranker can reorder candidates; if it is unavailable, fusion ranking remains the fallback. Context compilation selects bounded, query-relevant evidence while preserving provenance outside the generation prompt. Graph-RAG's integration endpoint requires a complete authorization scope. It excludes unscoped legacy records before dense/BM25 scoring; the host constructs that scope from the authenticated principal. [L1, L6, L12]

This is stronger than filtering forbidden documents after retrieval: a forbidden perfect match should never enter candidate scoring, prompt context, or citations. The exact behavior must still be exercised against a live, approved local/self-hosted corpus. The existing Graph-RAG storage configuration has not yet provided that final offline end-to-end evidence. [L2-L3]

2.3 Industrial diagnostics and deterministic calculations

The Time-Series Diagnostic Agent has a typed `diagnose()` interface and six domain packs: bearing, process, wind SCADA, battery, turbofan, and transformer. Its workflows distinguish detection, localization, diagnosis, verification, prognosis, uncertainty, and abstention. The host receives a structured diagnostic envelope; the model is not entrusted with safety-critical numerical calculation. Pump-102 evidence can include read-only telemetry, trends, asset context, and deterministic unit/threshold checks. [L1, L9]

Research status is mixed by domain. The frozen 95-event Wind CARE run reported recall 0.9111 but a normal-event false-alarm rate of 0.9400. Consequently Wind is excluded from platform demonstrations while a new, leakage-safe healthy-development calibration set is sought. The bearing program still needs specimen-level Paderborn comparisons and a held-out Lenze transfer test. These are important scientific blockers, not mere packaging tasks. [L9, L3]

2.4 ControlPlane and factuality verification

ControlPlane checks privacy, bias, hallucination/factuality, and conversation risk, then applies an explicit profile policy. The factuality subsystem extracts candidate claims, uses deterministic support/conflict checks, may escalate to retrieval plus a DeBERTa NLI scorer, and may use a bounded evidence-seeking agent. The v3 vocabulary separates SUPPORTED, CONTRADICTED, UNSUPPORTED, UNDECIDABLE, and CONFLICTING. A risk model is principally a verification-depth router, not proof that a claim is false. Claims and policy actions remain separate: a factuality status is evidence about an assertion; allow, warn, review, or block is a product decision. [L1-L2, L8]

There are three concrete gaps to close before claiming robust factual release control:

1. The heuristic claim extractor caps output at 40 claims; standard and deep NLI select only the first 8 or 20 eligible claims. The system does not currently prove that every material assertion was checked. [L8]
2. The customer-support profile defaults to allow and does not have a dedicated `claim_undecidable` rule. An uncertain claim can therefore be released when no other rule intervenes. [L8]
3. The evidence-quality gate may call evidence complete enough for an UNSUPPORTED finding after two provenanced chunks and a retrieval score threshold. Retrieval relevance is not proof that the authorized corpus was exhaustively searched. This path matters when optional support scoring is enabled. [L8]

No detector can prove unrestricted truth from incomplete or changing sources. The production objective is a measurable, calibrated **safe-release policy** with explicit uncertainty and human accountability. Atomic claims, evidence, and unknown states align with established fact-checking research, but method alignment alone is not accuracy evidence. [R1-R3]

2.5 Workflows, organizations, approval, and outputs

Versioned declarative Workcell Packs define trusted workflows and handlers; Pump Inspection v1.0.0 is included. Organization Packs express hierarchy, users, policies, assets, document manifests, and scoped Workcells without Python changes. Local identity combines role-based permissions with organization, workspace, department, classification, and explicit resource ACL checks. Requester/approver separation is enforced for business approval. Artifacts include actual DOCX, XLSX, and PPTX outputs with task/evidence lineage. [L1, L7]

These controls are strong prototype foundations. The local identity provider is not yet an enterprise IAM deployment, and draft CMMS behavior is not an authorization to alter plant state. [L1, L3]

2.6 Audit trail and Evidence Capsules

A completed Workcell run can assemble a capsule containing the final answer, input references, evidence fragments, calculations, claims, conflicts, workflow/model/policy manifests, tool calls, human decisions, audit events, and artifact copies. Each payload file is hashed with SHA-256. A canonical, sorted path/digest manifest yields the root identity. Independent verification recomputes file digests and root identity, detects missing or unexpected files, and can validate an optional Ed25519 signature against a trust store. The current model manifest can explicitly say `digest_verified=false`; a capsule does not automatically certify model identity. [L10]

The capsule proves **integrity of stored bytes and lineage**, not correctness of retrieved evidence, numerical interpretation, factual claims, authorization decisions, or human approval. Enterprise rollout also needs managed keys, retention, signed append-only audit, backup/recovery, and reviewed access policies. [L1, L3, L10]

3. What has actually been measured

3.1 Test and integration evidence

The project log records seven Pump-102 integration contract tests for authorized evidence, diagnostics, governance, outage, denial, provenance, artifact, and capsule behavior, plus 15 negative governance/resource/routing checks. These use simulated service boundaries and show expected failure semantics, not a live plant deployment. [L2]

For the latest controlled-inference engineering work, the focused protocol suite reported 83 passing tests. A full root backend run reported 241 passed and five failed; the unchanged base, tested in the same environment, had the same five failures. The documented causes include absent semantic-model dependencies and missing demo Markdown fixtures. Frontend typecheck and build passed in that recorded environment. These are historical documented runs; this PDF build did not rerun the complete suite. [L2]

3.2 Retrieval observations

On a small September 26 offline fixture, semantic Recall@1, Recall@3, MRR, and citation correctness were all reported as 1.0. Unanswerable refusal was 0.75 on four cases. A feature-hashing baseline reported Recall@1 0.812, Recall@3 0.938, and MRR 0.877. Query-aware compilation reduced one example from 444 estimated evidence tokens to 116 while retaining its required fact; a later authorized-answer prompt fell from about 1,356 to 708 tokens. These are **small fixture observations**, not an enterprise retrieval SLA. The local reranker was unavailable, so no measured reranker gain is claimed. [L2]

3.3 ControlPlane observations

An earlier integrated DeBERTa evaluation on 2,000 synthetic test cases reported 0.67% unsafe release versus 21.05% for a deterministic-only configuration. The integrated configuration also had 50.29% over-intervention, 68.00% action accuracy, 42.19% hallucination precision, 99.87% hallucination recall, and approximately 1,268 ms p95 latency. Labels were unreviewed synthetic candidates and source-family templates overlapped between development and test. Thus the result demonstrates a **tradeoff on generated variants**, not unseen industrial accuracy. [L8]

The later factuality-v3 result in the central project log reported 76.9% action accuracy, 66.3% contradiction precision, 97.6% recall, 78.9% F1, and about 12 ms p50 / 452 ms p95 factuality latency on legacy labels. That corpus predates the five-state taxonomy, so it cannot validate separate real-world precision for unsupported, undecidable, or conflicting claims. The two evaluations use different configurations and labels; do **not** present them as a clean before/after improvement. [L2]

3.4 Local inference observations and the incomplete new plan

The first Qwen3 0.6B smoke comparison completed 26/26 observations. A later preliminary controlled-live check contains 40 successful requests: 20 warm-ups and 20 measured requests, zero failed requests, and experiment_complete=false. It compared Windows Ollama with GGUF Q4_K_M against WSL CPU vLLM with BF16. Consequently it mixes runtime, weight format, and host differences. It lacks verified cold isolation, memory evidence, provider-verified identities, and independently reviewed quality labels. Neither run supports a claim that one runtime is generally faster or more accurate. [L2, L4]

The new v3 request-free plan contains 360 scheduled batches, zero requests, and experiment_complete=false. Its missing controls list includes identity, cold state, warm-ups, memory, required samples, token accounting, and quality labels. A plan is **not a benchmark result**. The harness now checks runtime and host identity, full prompt-token ceilings, randomized conditions, warm-up exclusion, reset evidence, process RSS/host RAM samples, and strict completion gates; the target-host behavior still requires live execution. [L1, L3-L4]

4. Equations and what they mean here

The equations below are either directly implemented in the cited module or recommended for evaluation. They are not substitutes for expert inspection of sensor units, time alignment, source revisions, or release decisions.

4.1 Implemented retrieval fusion

For a document chunk (d), the backend RRF score is

$$\text{RRF}(d) = \sum \text{over retrieved methods } m \text{ of } 1 / (k + \text{rank}_m(d))$$

where ($r_m(d)$) is its one-based rank in retrieval method (m); a method that did not retrieve the chunk contributes nothing. The class default is ($k=60$), and configuration can supply another positive value. RRF combines rank positions, not raw vector and BM25 scores. Authorization must precede either ranking method; otherwise a forbidden chunk could influence results even if it were later hidden. [L12, L6]

4.2 Illustrative engineering threshold calculation

The **synthetic, unreviewed** Pump-102 benchmark fixture states vibration ($x=8.2$) mm/s RMS and an alert threshold ($L=7.1$) mm/s RMS. Simple, unit-consistent arithmetic gives

$$\Delta = x - L = 1.1 \text{ mm/s RMS}$$
$$\delta = ((x - L) / L) \times 100\% \approx 15.5\%$$

This says the example measurement exceeds the example limit by 1.1 mm/s RMS, or about 15.5% relative to the limit. It does **not** by itself diagnose a bearing fault, justify shutdown, or authorize maintenance. Production use must confirm the source revision, instrument calibration, sampling window, asset identity, operating regime, and who can approve action. [L4, L1]

4.3 Recommended factuality and release metrics

For a predeclared target state, precision, recall, and F1 are

$$\text{Precision} = TP / (TP + FP) \quad \text{Recall} = TP / (TP + FN)$$
$$F1 = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

Report these **per state** (supported, contradicted, unsupported, undecidable, conflicting) and per risk slice. In addition, define a release-policy error with an explicit denominator:

$$\text{Unsafe release rate} = \text{released gold-hold cases} / \text{all gold-hold cases}$$

The historical report's own metric definition may differ; never splice values into this proposed denominator without checking its evaluator. Track over-intervention on gold releasable cases separately, plus review capacity, abstention rate, and p50/p95/p99 latency. A low observed error rate on a small sample is not proof of zero future errors. [L8, R3]

4.4 Implemented capsule identity

Let ($h_i=['\text{tex}', '\mathrm{SHA256}'](['\text{tex}', '\mathrm{bytes}'](f_i)))$. The capsule root is the SHA-256 hash of a canonical compact JSON array of sorted ($(['\text{tex}', '\mathrm{path}'], ['\text{tex}', '\mathrm{sha256}'])$) entries:

$$\text{Hroot} = \text{SHA-256}(\text{canonical JSON}(\text{sorted path / file-digest entries}))$$

The verifier checks both the stored files and this root. Integrity cannot prove that a statement inside a file was true. [L10]

5. Production blockers and why they matter

P0: No decision-grade end-to-end evidence. The networked Pump-102 run, approved local Graph-RAG corpus, live diagnostic evidence, and reviewed answer labels have not all been exercised together. Without this, a green component test can conceal a broken cross-service contract. [L2-L3]

P0: Factuality coverage and release gaps. Claim caps and ordered NLI limits can leave material assertions unverified; customer-facing undecidable claims do not have a dedicated policy action; retrieval relevance is treated too readily as evidence-set completeness. These should be fixed with tests before presenting ControlPlane as a dependable release gate. [L8]

P0: Scientific and evaluation leakage risk. The synthetic ControlPlane pack shares source families across splits. Wind CARE has a high normal-event false-alarm rate, and its consumed events cannot be reused as a supposedly fresh tuning set. Bearing validation requires specimen-level separation. Stronger models will not repair invalid labels or leaked splits. [L3, L8-L9]

P1: Deployment and security foundations. The repository uses local identity and SQLite prototype state. Enterprise IAM, service identity and authenticated service transport, document lifecycle, controlled secrets and keys, durable queues/events where needed, backup/restore, tamper-evident long-lived audit, incident procedures, and supply-chain provenance remain

planned. The root Compose file starts the main frontend/backend and optional Ollama/Qdrant profiles; it is not by itself evidence of an operated, multi-service production topology. [L1, L3, L13]

P1: Operational proof. Load, soak, failover, rollback, malformed input, outage, stale evidence, and recovery drills need target-host evidence. ControlPlane's recorded integrated CPU stress path had about 3.53 requests/second at concurrency four and 3,907 ms p95; deep verification needs explicit queue and latency budgets. [L8]

6. Step-by-step route to a production-quality supervised pilot

The phases below are proposed implementation work, **not completed features**. Each phase has an observable deliverable and a no-go condition. Use NIST's voluntary AI risk-management functions (govern, map, measure, manage) and Secure Software Development Framework as organizing references, not as certification claims. [R4-R6]

Phase 0 — Freeze scope and risk contract

1. Write an intended-use statement: authorized engineering assistance for maintenance analysis; read-only data; no plant writes or autonomous approval.
2. Name the accountable owner for document access, diagnostic thresholds, factual release, model changes, and incident response.
3. Define which errors are unacceptable: forbidden evidence entering ranking or context, a consequential unsupported recommendation being released, false completion when a service fails, or an unreviewed physical command.
4. Freeze a versioned system bill of materials: code revision, models and weights, prompts, policies, retrieval configuration, Workcell and Organization Packs, diagnostic version, and data snapshot.

Deliverable: intended-use statement, risk register, trust-boundary diagram, owner map, and explicit release/hold criteria.

No-go: unclear decision authority or any autonomous plant-write path. [L3]

Phase 1 — Close factuality and authorization gaps

1. Make extraction report coverage: response span coverage, claim overflow, skipped NLI claims, non-factual segments, and unsupported numeric/citation spans. Material unchecked text must trigger review or abstention.
2. Add explicit customer-profile behavior for UNDECIDABLE and verify all profile-by-state-by-consequential combinations with table-driven tests.
3. Remove chunk-count/relevance-score as a proof of evidence completeness. Accept an exhaustive-set signal only from a trusted, scoped retriever that records corpus version, query, access scope, time, and coverage method. Otherwise preserve UNDECIDABLE.
4. Require every releasable factual claim to retain a source ID, revision, evidence span, and access-scope lineage. Test stale, conflicting, forged, prompt-injected, and unauthorized-perfect-match evidence.
5. Recheck the host-to-Graph-RAG scope, Graph-RAG pre-ranking filter, host-to-ControlPlane evidence mapping, and final artifact/citation rendering together. Do not trust an LLM-generated citation as an access decision.

Deliverable: regression tests and machine-readable coverage report. **No-go:** a material unchecked or unauthorized claim can be emitted as verified. [L6, L8]

Phase 2 — Build an independent evaluation corpus

1. Use the existing plan's 300–500 industrial cases as an initial reviewed target, but treat sample size as an engineering start, not statistical certification. If real records are unavailable, build safely fabricated or redacted manuals, historian excerpts, and inspection cases with explicit synthetic provenance. Do not call them real.
2. For each case store query, response, each atomic claim and offsets, exact evidence bundle, access scope, document revision, expected factual state, expected policy action, and reviewer/date/rationale.
3. Include direct facts, units and numeric thresholds, multiple documents, contradictory revisions, missing evidence, stale evidence, cross-tenant perfect matches, long answers over claim limits, model timeouts, and adversarial instructions embedded in documents.
4. Use independent reviewers for ambiguous or consequential examples and adjudicate disagreement. Keep development, validation, and frozen test **source families** disjoint; do not repeatedly tune on the final holdout.

-
5. Add external reference datasets such as RAGTruth as a separate generalization check. It contains manually annotated RAG outputs but does not establish performance on the plant's domain. [R2-R3]

Deliverable: versioned data contract, split manifest, label guide, review records, and immutable holdout hash. **No-go:** labels invented by the same system being evaluated or source-family leakage. [L3, L8]

Phase 3 — Run a controlled measurement program

1. For retrieval, measure authorized Recall@k, citation/source correctness, denial leakage, stale-revision selection, unanswerable refusal, and per-stage latency. Compare dense, BM25, RRF, and optional reranking on identical query/corpus/access snapshots.
2. For ControlPlane, report a five-state confusion matrix, claim coverage, false SUPPORTED rate, unsafe releases, over-intervention, abstention, human-review load, calibration, and latency by profile and consequence.
3. For diagnostics, finish specimen-disjoint Paderborn physics/CNN/fusion ablations and independent Lenze transfer testing; obtain a separate healthy Wind development pool before revising Wind thresholds. Include abstention and operating-condition slices.
4. For inference, replace the synthetic quality fixture with genuinely reviewed labels and run the v3 protocol on the intended Windows/WSL hosts. Check all completion gates; never present a plan-only or partial run as a result. Compare GGUF-vs-BF16 as a combined format/runtime/host treatment, not a causal runtime effect.
5. Predeclare primary and guardrail metrics, acceptable error budgets, confidence intervals, latency/cost ceilings, and an independent go/no-go reviewer before examining frozen results.

Deliverable: reproducible reports, raw samples, code/data/model hashes, error clusters, and a signed evaluation decision. **No-go:** a headline metric without its denominator, holdout provenance, and failure slices. [L2-L4, L8-L9]

Phase 4 — Run the full Pump-102 assurance exercise

1. Stand up the approved local/self-hosted Graph-RAG store, diagnostic API, configured Ollama/vLLM runtime, ControlPlane, SovereignAI backend, and UI. Verify service identity, versions, private endpoints, health, and clock alignment before collecting timing evidence.
2. Ingest a reviewed, access-labeled Pump-102 evidence bundle. Run an authorized request through precheck, retrieval, diagnostic computation, local generation, release check, approval, artifact, audit, and capsule.
3. Verify each output claim against exact source/chunk and diagnostic IDs. Independently check units, thresholds, citation scope, approval separation, artifact content, and capsule root/signature.
4. Repeat with no evidence, deliberately unauthorized perfect-match evidence, stale/conflicting revisions, diagnostic abstention, model outage, Graph-RAG outage, ControlPlane outage, malformed replies, and timeout. Expected behavior is abstention, review, or explicit failure—not success.
5. Retain per-stage and total timings, error codes, response/citation snapshots, versions, and raw negative-case outcomes.

Deliverable: a replayable assurance bundle and a reviewable failure matrix. **No-go:** any required-service outage or denied evidence yielding a released confident recommendation. [L3, L5]

Phase 5 — Harden infrastructure and software delivery

1. Integrate enterprise identity and service-to-service authentication, least-privilege transport, centrally managed secrets, and rotation. Security-test tenant, department, workspace, and document ACL boundaries.
2. Add document lifecycle: deduplication, revision/supersession, reindexing, archival/deletion, quotas, and orphan cleanup. Version evidence snapshots so citations remain interpretable after updates.
3. Move process-local jobs/events and SQLite state to durable infrastructure only where the deployment's concurrency and recovery requirements demand it. Design migrations, idempotent retries, backups, restore tests, and an operator-approved rollback.
4. Pin dependencies and model weights; produce SBOM/build provenance, scan inputs/artifacts, test sandbox escapes, and establish signed releases and managed signing keys.
5. Add CI gates for unit, integration, ACL leakage, threat, migration, frontend, artifact/capsule, and offline model tests. Keep external egress checks and secrets out of test logs.

Deliverable: deploy/recover runbook with measured recovery exercise. **No-go:** untested restore, unauthenticated sensitive service, or a deployment that silently swaps model/policy/evidence identities. [L1, L3, R5]

Phase 6 — Shadow, supervise, then reconsider scope

- 1. Begin with read-only shadow evaluations: engineers continue the existing decision process while the system records proposed answers, uncertainty, latency, and disagreements without automating operations.
- 2. Review every consequential miss and a representative sample of routine outputs. Track drift in source revisions, model confidence, claim coverage, retrieval misses, false alarms, and user corrections.
- 3. Use canary rollout only after predeclared gates pass. Keep a known-good model/policy/corpus snapshot and a tested rollback switch. Assign incident owners and rehearse outage response.
- 4. Expand beyond advisory scope only through a **new** hazard analysis and explicit operator authorization; this report does not recommend plant writes or autonomous maintenance approvals.

Deliverable: supervised pilot report with residual risks and an accountable go/no-go decision. **No-go:** treating a quiet shadow period as proof of zero risk. [L3, R4-R6]

7. Immediate work order

The shortest useful sequence is: **(1)** fix ControlPlane claim-coverage and undecidable-release behavior; **(2)** replace the evidence-completeness shortcut and add negative tests; **(3)** prepare the reviewed, source-separated factuality and Pump-102 corpus; **(4)** make Graph-RAG storage local and prove pre-ranking authorization; **(5)** complete the target-host inference and diagnostic protocols; **(6)** run and archive the full networked Pump-102 success/failure matrix; **(7)** harden operations for a supervised pilot. This matches the existing project plan’s priority on an assurance bundle over new feature breadth. [L3]

No final production score is assigned here. The evidence is sufficient to describe the prototype and rank work, but not to quantify plant-level reliability, safety, security, or operational availability. NIST frameworks are useful process references; conformance and certification would require separate, applicable assessment. [R4-R6]

Source and terminology register

Local repository evidence

ID	Source and relevance
L1	docs/CURRENT_FEATURES.md — purpose, boundaries, implemented features, limits
L2	docs/PAST_EXPERIMENTS.md — historical tests, results, caveats
L3	docs/NEXT_PLAN.md — milestones, protocols, known gaps
L4	experiments/results/inference_v3_plan/summary.json and experiments/results/inference_controlled_live_check/summary.json — plan-only and preliminary states
L5	backend/app/integrations/orchestrator.py; tests/test_phase38_system_integration.py; tests/test_phase39_automatic_integration_routing.py — service sequence and contract tests
L6	Graph-RAG/README.md; Graph-RAG/server/server.ts — integration and scoped retrieval
L7	backend/app/identity/authorization.py; backend/app/workcells/; backend/app/artifacts/ — access, workflows, output
L8	Controlplane.ai/src/controlplane/checker.py; verification.py; evidence_quality.py; configs/policies/customer_support.yaml; src/adaptivefact/extraction/claim_extractor.py; docs/evaluation/COMPLETED_DATASET_EVALUATION.md — factuality design and measured limits
L9	Time-Series-Diagnostic-Agent/README.md; docs/PROJECT_DIRECTION.md — diagnostic domains, results, scientific gaps
L10	backend/app/capsules/builder.py; verifier.py; backend/app/identity/content.py — capsule construction and identity

ID	Source and relevance
L11	backend/app/sandbox/executor.py — Docker-only generated-code execution
L12	backend/app/rag/hybrid.py — implemented reciprocal-rank fusion
L13	docker-compose.yml — current local service topology

External primary references

- **R1.** Min et al., FActScore (EMNLP 2023), atomic-fact factuality evaluation: <https://aclanthology.org/2023.emnlp-main.741/>.
- **R2.** Thorne et al., FEVER (NAACL 2018), support/refute/not-enough-information evidence labels: <https://aclanthology.org/N18-1074/>.
- **R3.** Niu et al., RAGTruth (ACL 2024), manually annotated RAG hallucinations: <https://aclanthology.org/2024.acl-long.585/>; Wang et al., Factcheck-Bench (Findings EMNLP 2024), multi-stage fact-checker evaluation: <https://aclanthology.org/2024.findings-emnlp.830/>.
- **R4.** NIST AI Risk Management Framework 1.0, voluntary govern/map/measure/ manage structure: <https://www.nist.gov/itl/ai-risk-management-framework>. NIST notes that a revision is in progress as of this report date.
- **R5.** NIST SP 800-218, Secure Software Development Framework 1.1: <https://csrc.nist.gov/pubs/sp/800/218/final>.
- **R6.** NIST AI 600-1, Generative AI Profile: <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>.

Definitions. A *contract test* uses controlled doubles/fixtures to test the shape and failure behavior of an integration. A *holdout* is an evaluation set withheld from tuning. *Abstention* means the system declines to make a factual or operational conclusion when evidence is insufficient. *Unsafe release* must always be reported with its exact gold-label rule and denominator. *Production-quality* here means an operated, measured, reversible, and supervised advisory system—not a perfect detector or permission to control industrial equipment.